Merkle Tree-Based Inheritance in Decentralized Module Systems

S. C. Dunn
Starlabs, LLC
Colorado Springs, CO, USA
starlabsactual@gmail.com

Abstract

In this paper, we propose a novel method for module inheritance and dependency resolution in distributed systems using Merkle trees. This approach ensures deterministic finalization, secure content-based identification, and scalability for module registration and resolution. We address challenges such as the diamond problem, ambiguity in dependency resolution, and the need for decentralized, tamper-resistant module registries. By leveraging content-addressable hashing and authenticated data structures, our method enables efficient and secure inheritance verification. We provide formal definitions, proofs, and runtime analysis to demonstrate the feasibility of this approach in real-world decentralized systems. The method is implemented within the Tangl system, a distributed platform for managing decentralized modules, offering a flexible and secure foundation for module versioning.

CCS Concepts: • **Software and its engineering** → **Software configuration management and version control systems**; *Compilers*; *Distributed programming languages*; • **Theory of computation** → *Program semantics*; *Type structures*; • **Security and privacy** → *Cryptographic protocols.*

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Keywords: content-addressed modules, Merkle trees, decentralized registries, module resolution, module inheritance, type safety, generics, structural typing, inheritance, semantic versioning, reproducibility, distributed systems

ACM Reference Format:
S. C. Dunn. 2025. Merkle Tree-Based Inheritance in Decentralized Module Systems. In . ACM, New York, NY, USA, 27 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Module systems are fundamental to modern software development, enabling developers to create reusable components that can be seamlessly integrated into larger systems. In distributed environments, where modules can be authored, shared, and executed by diverse entities, managing dependencies and inheritance becomes increasingly complex. Traditional approaches to module inheritance often rely on fixed hierarchical structures or manual dependency resolution, leading to ambiguity and inefficiency [3].

In decentralized systems, the necessity for a reliable, transparent, and scalable method to handle module dependencies is paramount. Centralized dependency resolution systems face limitations, including performance bottlenecks, trust issues, and challenges with versioning and compatibility [6]. Moreover, the absence of a single authority for managing dependencies heightens the risk of conflicts, version mismatches, and errors in module resolution. In the absence of clear guarantees, developers often resort to manual dependency resolution or depend on unreliable third-party services.

Merkle trees offer a robust solution to these challenges by providing a cryptographically secure method to verify content and resolve dependencies in a distributed setting. Merkle tree-based inheritance of modules using content addressing facilitates a scalable and verifiable approach to determining module relationships, allowing modules to inherit behavior and data from other modules in a secure and deterministic manner. This paper presents a Merkle tree-based inheritance model that addresses these issues, ensuring predictable, conflict-free inheritance resolution.

We define modules generically in this paper, though a specific realization of this model (including the Knot abstraction) is detailed in the Tangl project [10].

2 Background

Managing module inheritance and dependencies in decentralized systems is a complex challenge. In such systems, modules are typically authored and maintained by independent parties, which complicates traditional mechanisms of dependency resolution. Unlike centralized systems where a single authority can enforce consistency, decentralized systems require a more flexible, verifiable, and autonomous method of resolving dependencies and managing module inheritance.

Traditional inheritance models, such as those used in object-oriented programming (OOP), assume a fixed, centralized hierarchy [13]. These models do not scale well in distributed environments, where the dependencies and inheritance structures are more fluid and subject to change. Furthermore, the centralized nature of traditional inheritance can create bottlenecks, as all module resolutions pass through a single point of failure. In contrast, decentralized systems require solutions that allow for module resolution and dependency tracking without relying on a central authority.

In content-addressed distributed systems, modules are identified by cryptographic hashes rather than names or locations, which makes traditional dependency resolution strategies impractical. The system must ensure that all parties have a consistent view of the module graph, even as modules evolve over time. Merkle trees provide an elegant solution for this challenge by enabling efficient and secure verification of the contents and relationships between modules [16]. Merkle trees allow for modular and hierarchical verification: by hashing both the contents of a module and its relationship to other modules, Merkle trees ensure that the integrity of the system can be verified with minimal computational overhead.

Module inheritance in such a system refers to the process by which new modules can be derived from existing ones, with the ability to reuse, extend, or override functionality from parent modules. This inheritance is not managed through a fixed hierarchy but is instead encoded in the module's content hash, providing a decentralized, flexible, and verifiable method for composing new modules. By leveraging content addressing and Merkle trees, inheritance in this context is automatically traceable, immutable, and resistant to tampering.

The use of Merkle trees for inheritance also facilitates the implementation of distributed hash tables (DHTs) and other decentralized lookup methods, ensuring that module relationships and dependencies can be resolved efficiently without central coordination. This approach supports greater scalability, fault tolerance, and security in distributed systems, making it a promising solution for managing module inheritance in large-scale decentralized applications.

3 Related Work and Motivation

Several decentralized systems, such as IPFS, Git, and package managers like npm and Cargo, have addressed aspects of content-addressable storage and versioning [2][5][7][18]. These systems use cryptographic hashes to ensure the integrity of modules but do not provide a formalized mechanism for handling inheritance or dependencies. This paper builds upon these systems by introducing a Merkle tree-based inheritance model, where module dependencies and inheritance relationships are captured in a cryptographically secure and deterministic manner.

Existing decentralized dependency systems like Ethereum's smart contracts provide basic inheritance models, but they often suffer from issues such as the diamond problem, where multiple inheritance paths can create ambiguity [11][20]. Our approach resolves this by leveraging Merkle trees to explicitly define the resolution order for every inherited symbol, guaranteeing determinism and consistency across all executions.

4 Challenges Addressed by Merkle Tree-Based Inheritance

The Merkle tree-based inheritance model employed in this system addresses several classical challenges associated with inheritance and module composition. The following subsections describe how this model resolves these issues through cryptographic hashes, content-addressable structures, and canonical ordering.

4.1 The Diamond Problem

A common challenge in multiple inheritance systems is the diamond problem, where a module inherits from two parents that themselves share a common ancestor [20]. This can cause ambiguity in the application of inherited logic or data.

In the Merkle inheritance model, all parent modules are identified by their content hash. The ancestry tree is computed using a topological traversal, and duplicate content hashes are merged. Thus, a shared ancestor is represented once in the Merkle tree, eliminating redundant initialization or conflicting resolutions.

For example, consider the following inheritance structure:

$$\text{Module } F \to \{D, E\}, \quad D \to \{A, B\}, \quad E \to \{A, C\}$$

The module A will appear only once in the computed Merkle tree of F, ensuring that all inherited elements from A are unified without duplication or ambiguity.

4.2 Ambiguity in Parent Resolution

In traditional inheritance models, ambiguity can arise when multiple inheritance paths lead to similar or conflicting methods or fields, often requiring manual disambiguation [12]. In contrast, Merkle tree-based inheritance defines a **canonical order** for ancestry resolution that eliminates ambiguity through a deterministic, verifiable process. The order is determined by the following principles:

- **Lexicographic ordering of content hashes**: The content hashes of modules are used to establish a consistent order, ensuring that modules with the same ancestry are resolved based on the lexicographical comparison of their hashes.
- **Topological sorting of the directed acyclic graph (DAG)**: The directed acyclic graph formed by module imports is topologically sorted to respect the dependency relationships between modules. This guarantees that a module's ancestors are processed in a consistent, hierarchical order.
- **Optional rule-based tie-breaking**: In cases where modules are equally prioritized in the lexicographic or topological order, tie-breaking can be applied using rules such as depth-first priority or timestamp-based preference. These rules provide additional determinism and help resolve any conflicts when multiple paths lead to the same module.

This canonical order ensures that each module's ancestry is resolved in a unique and consistent manner, promoting both predictability and verifiability in the resolution process.

4.3 Cyclic Dependencies

The use of content addressing and Merkle trees inherently prevents cyclic inheritance, ensuring a well-structured and acyclic dependency graph [2]. Since each module's identity is determined by the content hashes of its declared imports, the formation of a cycle would create a circular hash dependency. This is mathematically impossible due to the properties of cryptographic hash functions such as SHA-256, which are one-way functions and inherently acyclic. As a result, any attempt to form a cycle in the module graph would result in an invalid state, ensuring that dependencies remain acyclic and verifiable.

4.4 Tamper Resistance and Integrity

Each finalized module inherits not only behavior but also a cryptographically verifiable proof of ancestry. Because the Merkle tree includes all transitive ancestors in a tamper-proof structure, any modification to an ancestor will:

- Alter the content hash.
- Invalidate the Merkle root.
- Propagate changes upward, altering all dependent modules.

This ensures referential transparency and immutability across distributed environments.

4.5 Efficient Validation and Caching

Once a module is finalized, its computed Merkle tree and hash can be persisted. This enables downstream modules to:

- Reuse validated ancestry trees without recomputation.
- Quickly verify conformance and inheritance during finalization or execution.
- Improve scalability in distributed systems by caching shared subtrees.

The use of content-addressable structures ensures that identical inheritance trees yield identical hashes, supporting efficient distributed caching and deduplication.

5 Comparison with Existing Systems

The module resolution model described in this paper, built on symbolic references, lexical shadowing, and content-addressed identifiers, marks a substantial departure from existing approaches to software composition. Traditional systems such as `npm`, `Cargo`, and `pip` resolve dependencies through a centralized registry and a version constraint solver, while reproducibility-focused systems like Nix and IPFS use content hashes and store-path resolution. By contrast, our model introduces a novel runtime-based resolution strategy based on deterministic symbol lookup over a hierarchical, immutable reference graph.

These differences are reflected in our prototype implementation, *Tangl*, which operationalizes the model in a distributed setting. In Tangl, modules reference other modules via symbolic names bound to content hashes in a reference map. The resolution of a symbol is deferred until a dedicated resolve phase during runtime, during which all transitive bindings are located, either from local cache or the network. This deferred resolution strategy enables lazy loading, caching optimizations, and precise control over symbol scope.

Unlike version-based systems, which use constraint solvers to select from a range of candidate versions, our model treats resolution as a lexical and symbolic process. The order and specificity of imports determine which symbol is selected. For example, a module that imports `b` followed by `a`, then calls `foo()`, will resolve the call to `b.foo`, unless `a.foo` was explicitly imported using a more granular import like `import(a.foo)`, in which case `a.foo` takes precedence. This mechanism eliminates ambiguity and avoids the need for version negotiation or lockfiles, while still supporting fine-grained override and shadowing semantics.

The resolve phase, as implemented in Tangl's runtime, recursively traverses the reference map to locate all required content-hashed modules. This traversal is deterministic and requires no external coordination, ensuring reproducibility and verifiability. Once all required modules are located, execution can proceed with a complete and locally available resolution environment. This allows Tangl to function efficiently in distributed and offline contexts without sacrificing correctness or transparency.

By treating linking and resolution as symbolic substitution within a content-addressed graph, the model eliminates many of the ambiguities and global coordination problems that arise in versioned dependency systems. The Tangl prototype demonstrates how this model can be implemented in practice, offering a viable path forward for verifiable, distributed software composition.

6 Merkle Tree-Based Inheritance Model

6.1 Merkle Trees

A Merkle tree, or hash tree, is a binary tree in which each leaf node represents a hash of a data block, and each non-leaf node represents a hash of its children. This structure allows for efficient and secure verification of the integrity of large data structures. In the context of module inheritance, Merkle trees can be used to represent the relationships between modules and their dependencies.

6.2 Content-Addressed Module Systems

We define a content-addressed module system as a module system in which each module is uniquely identified by a cryptographic hash of its content and its declared dependencies. Unlike traditional systems that rely on names, versions, or centralized registries, content-addressed systems derive identity directly from the structure and substance of the module.

This approach enables several key properties essential to decentralized environments:

- **Immutability:** Modules cannot be altered without changing their identity.
- **Tamper Resistance:** Integrity can be verified independently by re-computing hashes.
- **Reproducibility:** Module graphs are guaranteed to resolve the same way on any machine.
- **Decentralized Resolution:** Modules can be retrieved and validated without reliance on a central authority.

In this paper, we use the term *content-addressed module system* to refer to any system that satisfies these properties and supports deterministic, verifiable module inheritance.

6.3 Module Representation

In our model, each module is represented as a leaf node in the Merkle tree structure. The module's content, including its code, metadata, and declared dependencies, is cryptographically hashed to generate a unique identifier, known as the *module ID*. This identifier is derived from the module's content-addressed structure, meaning that any modification to the module, whether it's a change in code, metadata, or dependencies, will result in a different hash and, consequently, a different module ID.

This content-addressed approach guarantees the integrity of the module, as any change in its content will be reflected in the hash. Additionally, it provides traceability, as the history of changes to the module can be traced back through its hash history in the Merkle tree. The Merkle tree structure ensures that these changes are efficiently verifiable and can be resolved deterministically across distributed systems.

6.4 Inheritance Mechanism

Inheritance in our model is represented by linking modules through their Merkle tree hashes. A module can inherit from another by referencing the parent module's content-addressed ID. This reference establishes a clear and verifiable relationship between modules, ensuring that all dependencies are properly accounted for and that the inheritance hierarchy remains consistent. The use of Merkle hashes ensures that any change in the parent module's content is reflected in the child module's identity, providing strong guarantees of integrity and traceability across the entire inheritance tree.

6.5 Canonicalization of Inheritance Trees

Let each module import zero or more parent modules via symbolic import paths. We define an *ancestry DAG* $\mathcal{G}_M = (V, E)$, where V are module content hashes and E are edges representing declared imports.

To compute a canonical inheritance sequence:

1. Construct $\mathcal{G}_M$ transitively from all imports.

2. Topologically sort $\mathcal{G}_M$, using content hash order as a deterministic tie-breaker.
3. Traverse the resulting DAG and apply a fixed-point de-duplication.

Formally, we define the linearized ancestry list $\mathcal{L}_M$ of a module M as:

$$\mathcal{L}_M = \text{fix}(\lambda L.\text{dedup}(\bigcup_{\text{parent} \in M.\text{imports}} \mathcal{L}_{\text{parent}} \cup \{M\}))$$

This process produces a content-addressed, deterministically ordered, duplicate-free ancestry list. Cyclic imports are prevented by the deterministic and immutable nature of the finalization process.

6.6 Clarification of Inheritance Semantics and Edge Cases

While the formal model for inheritance in Merkle tree-based systems provides a sound theoretical foundation, it is crucial to address practical scenarios, especially edge cases like circular dependencies or conflicting module versions.

6.6.1 Circular Dependencies. In our model, circular dependencies are inherently prevented by the resolution process. Since the system relies on content-addressed resolution and Merkle tree-based linking, each module must resolve all its symbols before finalization. If a cycle were to occur, one of the modules in the cycle would fail to resolve its symbols, as it would depend on the unresolved module, preventing the cycle from completing.

This ensures that the module resolution process remains acyclic and predictable, with each module being independently resolvable before it is finalized and added to the module graph. Any attempt to resolve a cycle would trigger an error or warning to the developer, who would need to break the circular dependency manually, ensuring that the graph remains valid.

6.6.2 Conflicting Module Versions. Conflicts arise when different modules depend on different versions of the same dependency. In traditional systems, this often results in version resolution errors or unexpected runtime behavior due to global state or shared environments. In contrast, our content-addressed inheritance model safely supports multiple versions of the same module coexisting within the same dependency graph.

Because each module is identified by a cryptographic content addressing hash, different versions naturally produce distinct, immutable identifiers. This ensures that even if multiple modules reference different versions of the same dependency, their imports remain isolated and unambiguous. The system does not attempt to merge these dependencies unless explicitly directed to do so by the developer.

When modules are executed in sandboxed runtimes, such as WebAssembly instances, this isolation extends to the runtime level, preventing symbol collisions and ensuring that internal state and behavior do not interfere across versions. Tools can further aid developers by exposing precise version paths during resolution, enabling awareness of transitive dependency trees and allowing version pinning or override declarations.

For example, two versions of the same module can be imported under separate names:

```
import "crypto@1.2.0" as crypto_v1;
import "crypto@2.0.1" as crypto_v2;
```

By embracing this explicit, hash-based strategy, our system provides strong guarantees of reproducibility and safety, even in the presence of complex or divergent dependency graphs. Developers retain full control over which versions are used where, and tooling can report potential conflicts, overrides, or redundancies for better maintainability.

This approach makes version conflicts not a source of ambiguity, but an intentional, traceable design choice, supporting both fine-grained control and large-scale modular reuse.

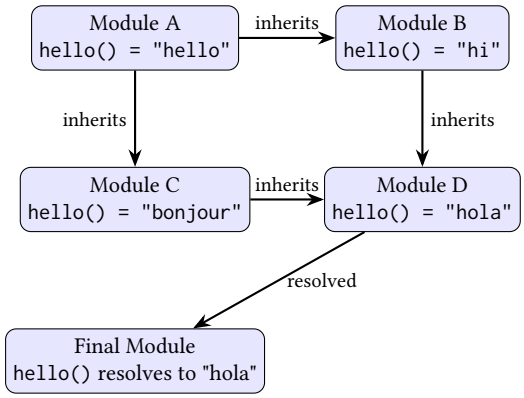


Figure 1. Merkle-based inheritance with multiple overrides. Module A defines `hello()`, which is overridden in B and C. D inherits from both B and C, and the final module resolves `hello()` from D.

6.7 Formal Module Semantics

The semantics of Merkle tree-based inheritance draw on foundational work in the semantics of module systems [8], adapting traditional constructs such as imports, exports, and substitution to a decentralized, content-addressed setting. Each module is treated as a unit of encapsulation and composition, with Merkle roots representing immutable snapshots of the module's symbol table and inheritance relationships forming a directed acyclic graph (DAG).

To reason about soundness and determinism, we model symbol resolution as a total function over a Merkle DAG, where shadowing and override precedence follow a deterministic merge strategy akin to linear extension of the ancestry graph. This is similar in spirit to mixin module semantics [1], where module composition preserves override order and type safety. Our system ensures that for any well-formed module graph, symbol resolution is unambiguous and stable under extension, key properties necessary for compositionality.

We define a symbolic resolution trace as a path from the querying module through its ancestry DAG to the module that defines a symbol. This trace is unique (due to deterministic merge rules) and reproducible (due to hash-based identity), enabling both static reasoning and runtime validation.

Future work will extend this semantic model into a formal calculus for decentralized modules, allowing proofs of compositional soundness, override determinism, and evaluation invariance over distributed execution contexts.

6.8 Real-World Use Cases and Interoperability

To illustrate the practicality of Merkle tree-based inheritance, we present a series of real-world development scenarios that highlight the system's resolution semantics, support for interoperability, and improvements over existing dependency systems.

Use Case 1: Overriding and Forking a Dependency. Suppose a developer uses a module `libA` that defines a function `validate()`. In a traditional ecosystem like npm or Cargo, modifying `validate()` requires forking the entire package and updating downstream dependencies manually. In contrast, our system allows a new module `libB` to inherit from `libA` and override only the `validate()` function. All unchanged behavior remains inherited by hash, and tools can trace the exact override provenance via ancestry inspection. This reduces friction when applying custom patches without sacrificing reproducibility.

Use Case 2: Semantic Version Resolution and Module Inflation. Semantic versioning (semver) in npm allows flexible version specifiers (e.g., `^1.2.3`), often resulting in projects resolving to multiple versions of the same package. This can bloat dependency trees and cause non-deterministic behavior [19]. In our system, semver references are resolved to content hashes before finalization, ensuring that the final build is content-addressed and deterministic. Each version resolution is recorded explicitly, enabling debugging and reproducibility, even when upstream modules evolve.

Use Case 3: Integration with npm, Cargo, and pip. To bridge existing ecosystems, we define import adapters for package managers such as npm, Cargo, and pip. These adapters translate semver-based imports into finalized Merkle roots at the time of module finalization (See Figure 2). For instance:

Listing 1. npm-style import resolved to content hash

```
"dependencies": {
  "left-pad": "^1.3.0"
}
```

is resolved via a Merkle-aware registry to:

```
"dependencies": {
  "left-pad": "merkle:9a3f...12e7"
}
```

Unlike traditional systems, this transformation is not stored in a lockfile. Instead, the finalized Merkle root of the module captures the complete and immutable dependency graph. This ensures reproducibility without introducing auxiliary metadata files and maintains full compatibility with deterministic builds and ancestry tracing.

Protocol for Handling Version Mismatches. Content addressing enables multiple versions of the same module to coexist safely in the same dependency graph, as each version is identified by a unique hash. This removes the traditional constraint of globally singular versions and allows different ancestors to reference distinct versions without collision. When necessary, developers may intentionally override transitive dependencies by introducing an explicit wrapper module that inherits from the conflicting versions and reconciles any behavioral or interface discrepancies. These override semantics make conflict resolution composable and transparent, allowing developers to control shadowing behavior, expose specific symbols, or patch inconsistencies at defined boundaries. This approach eliminates ambiguity, avoids silent breakage, and supports reproducible and observable dependency graphs.

7 Formalization of Finalization

Finalization refers to the process of resolving a module's dependencies and determining its final state. In our model, finalization is deterministic and injective, meaning that for a given set of inputs, there is a unique final state.

7.1 Determinism

Determinism ensures that the final state of a module is predictable and consistent. Given the same inputs, the finalization process will always produce the same result.

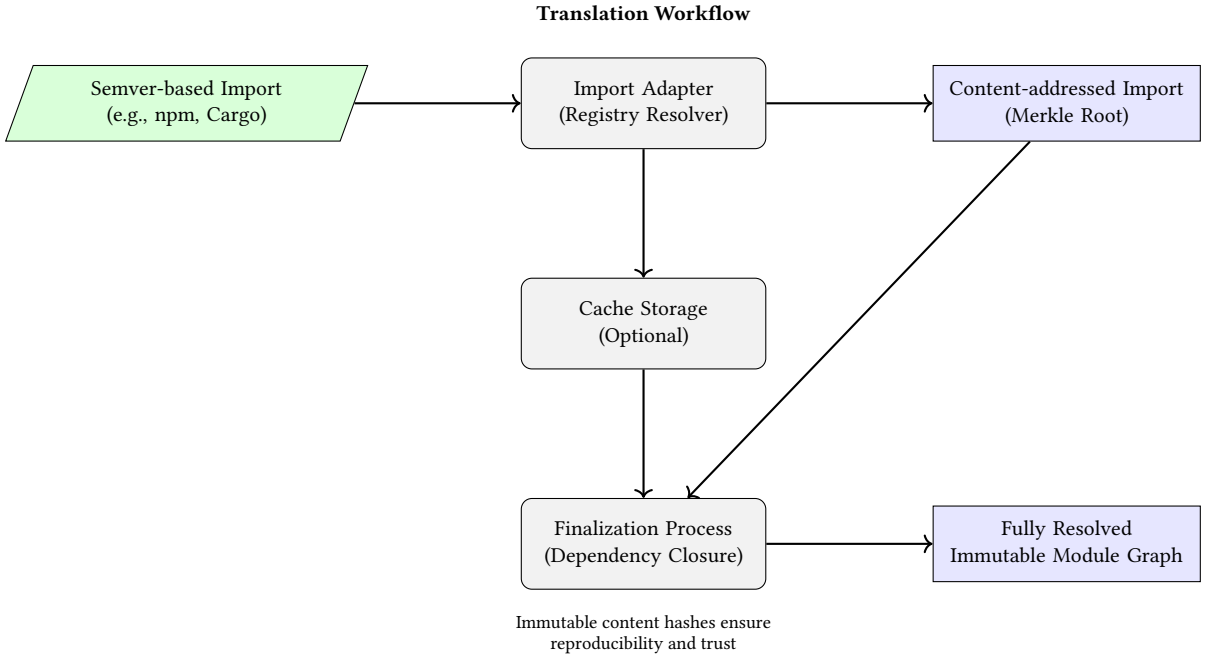


Figure 2. Workflow for translating semver-based imports into Merkle-rooted content-addressed modules. Workflow for translating semver-based imports into Merkle-rooted content-addressed modules. Semver-based imports from npm or Cargo are read into an import adapter that does registry resolutions. The imports are resolved to content-addressed imports, optionally using a cache for resolution. The content-addressed imports are passed into the finalization process that handles dependency closure. The result is a fully resolve immutable module graph.]

This property is crucial for debugging and maintaining distributed systems.

7.2 Injectivity

Injectivity guarantees that each unique set of inputs results in a unique final state. This property prevents conflicts and ambiguities in module resolution, ensuring that modules are correctly and uniquely identified.

7.3 Modules as Values

In our model, a finalized module is treated as a first-class value with a well-defined structure. Each module is represented as a tuple:

$$M = (C, T, A, P)$$

where:

- C denotes the module's content, which may be source code, bytecode, or any other serializable data.
- T is a UNIX timestamp that records the moment of finalization (i.e. publication).
- A is the author's identifier, typically represented as a public key or cryptographic identity.
- $P = \{M_1, M_2, \ldots, M_n\}$ is the set of parent modules from which M inherits. All parents must themselves be finalized prior to inclusion.

This tuple encapsulates both the internal semantics and external context of a module. The structure enables consistent hashing, resolution, and identity derivation across distributed systems. Throughout our semantics, we assume the presence of a cryptographic hash function $H(\cdot)$, for example, SHA-256, to ensure content-addressable integrity.

7.4 Module Identity and Content Hashing

The identity of a module is determined by both its internal content and its external context. To begin, we define a **content hash** that captures the internal semantics of the module independently of its ancestry:

$$h_C = H(C)$$

where C represents the module's source content (including declarations, definitions, and symbol exports), and H is a cryptographic hash function such as SHA-256. This ensures that any module with identical internal structure will produce the same h_C, regardless of its author or inherited dependencies.

To establish the final, globally unique identity of the module, we incorporate the content hash alongside its full provenance:

$$h_M = H(h_C \,||\, T \,||\, A \,||\, H_{\text{merkle}}(P))$$

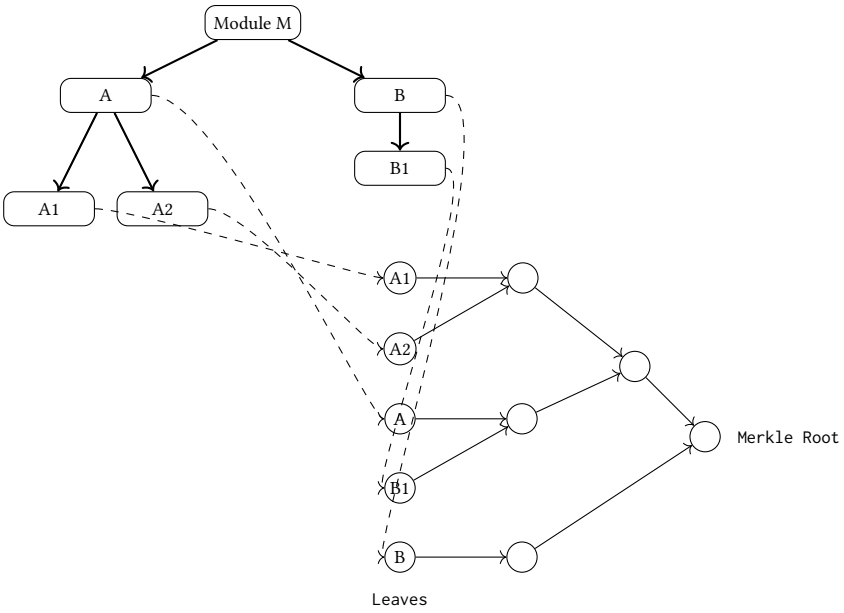


Figure 3. Mapping of transitive ancestors to Merkle leaf nodes and internal hash structure.

Here, T is a timestamp or logical clock, A is the author's identifier (such as a public key or registry signature), and $H_{\text{merkle}}(P)$ is the Merkle root of the module's transitive ancestry graph.

Together, these components ensure that changes to a module's source code, ancestry, or metadata produce a distinct identity. This structure guarantees reproducibility, supports immutable caching, and underpins decentralized inheritance and resolution in content-addressed module systems.

7.5 Ancestry and Merkle Root

The ancestry of a module is defined as the transitive closure over its parent set:

$$A(M) = \bigcup_{m \in P} \{m\} \cup A(m)$$

To compute the ancestry root (See Figure 3):

1. Collect the final IDs h_{m_i} of all ancestors in $A(M)$.
2. Sort them lexicographically.
3. Construct a balanced binary Merkle tree from this sorted list.
4. Let $H_{\text{merkle}}(P)$ be the root hash of the resulting tree.

This ordering guarantees deterministic ancestry representation and allows compact proofs of inclusion.

7.6 Symbol Resolution

Each module M exports a set of named symbols:

$$S_M = \{s_i = (\text{name}, \text{value}, \text{type})\}$$

During finalization, M computes a static resolution table:

$$R_M : \text{name} \to \langle M_i, s_j \rangle$$

For each symbol name, the table identifies the ancestor $M_i \in A(M)$ and the symbol s_j it resolves to. Resolution is based on the canonical Merkle ordering of ancestors and is computed once, at finalization time.

7.6.1 Formal Resolution Rules.

We define a judgment:

$$M \vdash s \Downarrow d$$

meaning that in module M, symbol s resolves to definition d.

$$\frac{s \in \text{dom}(\Gamma_M) \qquad \Gamma_M(s) = d}{M \vdash s \Downarrow d} \quad \text{(Local)}$$

$$\frac{P_i \vdash s \Downarrow d \qquad \forall j < i.\ s \notin \text{dom}(\Gamma_{P_j})}{M \vdash s \Downarrow d} \quad \text{(Inherit)}$$

7.6.2 Override Traces.

We extend the resolution relation with a symbolic trace π of visited modules:

$$M \vdash s \Downarrow d \rightsquigarrow \pi$$

$$\frac{s \in \text{dom}(\Gamma_M)}{M \vdash s \Downarrow d \rightsquigarrow [M]} \quad \text{(T-Loc)}$$

$$\frac{P_i \vdash s \Downarrow d \rightsquigarrow \pi \qquad \forall j < i.\ s \notin \text{dom}(\Gamma_{P_j})}{M \vdash s \Downarrow d \rightsquigarrow [M] \mathbin{+\!\!+} \pi} \quad \text{(T-Inh)}$$

7.6.3 Determinism Guarantee.

Theorem 7.1 (Determinism). *If $M \vdash s \Downarrow d_1$ and $M \vdash s \Downarrow d_2$, then $d_1 = d_2$.*

Sketch. By construction of the resolution chain: s resolves to the first ancestor providing it, uniquely determined by Merkle-ordering. Resolution is a function. □

7.7 Typing and Soundness

To support type-safe composition of modules, we define a minimal static type system over symbol declarations. Each symbol is assigned a type τ, and a module typing context Γ maps fully qualified symbol paths to their types.

Typing Judgment. We write $\Gamma \vdash m : \text{OK}$ to mean that module m is well-typed under environment Γ. This judgment checks that all symbol definitions and references are type-consistent, and that overrides respect declared types in inherited modules.

Subtyping. We allow structural subtyping between types. Let $\tau_1 <: \tau_2$ denote that τ_1 is a valid refinement of τ_2, allowing overridden definitions to narrow their types while preserving substitutability. The subtyping relation is assumed to be reflexive and transitive.

Override Soundness. During finalization, when a symbol s defined in a child module overrides a symbol of the same name in a parent module, we require that its type be a subtype of the parent definition:

$$\text{If } \Gamma(p.s) = \tau \text{ and } \Gamma(c.s) = \tau', \text{ then } \tau' <: \tau$$

This ensures that the overriding definition is type-compatible with its use sites in the parent module, preserving module-level soundness.

Type Preservation. Finalization preserves typing under symbol resolution. If a module m finalizes to a Merkle root r, and $\Gamma \vdash m : \text{OK}$, then all resolved symbols in r are assigned types that satisfy the subtyping constraints imposed by inheritance. That is, for every symbol s in the finalized module, there exists a unique τ_s such that $\Gamma(s) = \tau_s$ and all overrides respect τ_s up to subtyping.

Discussion. This type system is intentionally minimal to support early experimentation and accommodate a wide range of module languages. More expressive typing features, such as recursive types, generics, or dependent types, can be layered on top of this model. For ecosystems with strong static typing (e.g., Rust or TypeScript), future work may define language-specific typing extensions that plug into this framework.

Example: Function Signature Subtyping. Suppose a parent module defines a symbol `process` with type:

$$\Gamma(\texttt{parent.process}) = \texttt{String} \to \texttt{Int}$$

A child module may override this symbol with a more specific implementation:

$$\Gamma(\texttt{child.process}) = \texttt{NonEmptyString} \to \texttt{Int}$$

Assuming `NonEmptyString` is a subtype of `String`, the override is valid since the new function accepts a narrower input type. This contravariant rule for function inputs ensures substitutability: wherever the parent's `process` was expected, the child's version can safely be used.

Likewise, return types follow covariance. If a function originally returns a type `Animal`, and an override returns `Dog`, the substitution is also valid:

$$\texttt{Animal} \leftarrow \texttt{Dog} \quad (\text{i.e., } \texttt{Dog} <: \texttt{Animal})$$

Together, these rules preserve soundness under function subtyping in both directions of the signature.

7.8 Named References and Registry Resolution

Before finalization, parent references may be symbolic, e.g.:

```
import math/utils@1.2.0
```

These are resolved by a registry function:

$$\mathcal{R} : (\text{name}, \text{semver}) \to h_{M'}$$

Once resolved, symbolic references are replaced by concrete module hashes and included in P, preserving all cryptographic guarantees.

7.9 Finalization Algorithm

Finalization is a total function:

$$\mathcal{F}(C, T, A, \{r_1, \ldots, r_n\}) = h_M$$

Where each r_i is either:

- a finalized module ID h_{M_i}, or
- a symbolic reference to be resolved through $\mathcal{R}$.

Finalization proceeds as:

1. Resolve all symbolic references via $\mathcal{R}$.
2. Ensure all h_{M_i} are finalized and valid.
3. Compute $A(M)$ and derive the sorted list of ancestor IDs.
4. Compute $H_{\text{merkle}}(P)$ as the Merkle root over that list.
5. Hash the module content: $h_C = H(C)$.
6. Compute the final module ID:

$$h_M = H(h_C \,||\, T \,||\, A \,||\, H_{\text{merkle}}(P))$$

This procedure is deterministic and self-contained.

7.10 Determinism and Injectivity

Let $X = (C, T, A, P)$ be the full input to $\mathcal{F}$.

Then:

Determinism. : $\mathcal{F}(X)$ always returns the same output for the same input.

Injectivity. : if $\mathcal{F}(X_1) = \mathcal{F}(X_2)$, then $X_1 = X_2$.

Together, these properties ensure that the identities of the final modules are both predictable and collision-resistant.

7.11 Formal Proofs for Determinism and Injectivity

We establish that the module finalization process is both deterministic and injective, ensuring that content-addressed identities are stable and unambiguous.

Determinism. Let a module be defined as a tuple $X = (C, T, A, P)$, where C is the content, T is the timestamp, A is the author's identifier, and P is the set of finalized parent modules. The final module identity h_M is computed as:

$$h_M = H(h_C \,||\, T \,||\, A \,||\, H_{\text{merkle}}(P))$$

where $h_C = H(C)$ is the content hash and $H_{\text{merkle}}(P)$ denotes the Merkle root over the ancestry set P. Because all components are passed through deterministic functions (e.g., cryptographic hash functions), the result is fully determined by the inputs. Thus, finalization is deterministic: repeated computation with the same input always produces the same output.

Injectivity. Injectivity guarantees that no two distinct modules produce the same finalized identity. Formally, if:

$$\mathcal{F}(X_1) = \mathcal{F}(X_2)$$

then it must follow that:

$$X_1 = X_2$$

This follows from the collision-resistance of the cryptographic hash function H, and from the fact that the identity h_M includes all distinguishing information: module content, creation timestamp, author identity, and complete ancestry. Any change to any component will yield a different hash, ensuring that the finalization function $\mathcal{F}$ is injective.

Together, these properties establish the basis for reliable, reproducible, and verifiable module resolution in content-addressed systems.

7.12 Runtime Linking

Each finalized module M carries its resolution map R_M.

When the runtime encounters a symbol reference, such as:

`call(`f`)`

, it performs:

$$R_M(f) = \langle M_i, s_j \rangle$$

The VM then loads s_j from M_i's exports. Because R_M is precomputed, symbol resolution during execution is unambiguous and fully determined by the module's final identity h_M.

7.13 Compositionality and Determinism in Decentralized Resolution

Our resolution semantics ensure that symbol lookup in decentralized, content-addressed module graphs is both compositional and deterministic, even in the presence of adversarial network conditions or partial replication.

Compositionality. We define compositionality as the principle that the semantics of a module composed from multiple parents is determined entirely by the semantics of its parents and their declared merge order. Specifically, if module M inherits from modules A and B, the resolution behavior of M is fully derived from the resolution semantics of A and B, combined with the override graph defined by the inheritance declarations. No additional global context is required.

Determinism. Symbol resolution is deterministic when a given symbol in a finalized module graph always resolves to a unique, well-defined definition. Because all dependencies are identified by cryptographic content hashes, resolution operates over immutable structures, eliminating ambiguity introduced by mutable global state, varying dependency trees, or inconsistent caching.

Resolution Semantics. Resolution proceeds via a recursive traversal of the Merkle DAG, honoring an explicitly declared left-to-right parent merge order. Conflicts are resolved according to a fixed override rule: later parents override earlier ones unless explicitly merged or shadowed. This process is formalized as big-step operational semantics (Figure 4), where a symbol resolution trace corresponds to a unique path in the ancestry graph. Each step in the trace is locally verifiable using only the content-addressed module identifiers.

As future work, we plan to mechanize these resolution semantics in a proof assistant such as Coq or Lean, to establish the following metatheoretic properties:

- **Soundness:** Every resolved symbol corresponds to a concrete definition in the ancestry graph.
- **Determinism:** A symbol always resolves the same way across all valid executions.
- **Compositionality:** Substituting a module into a larger graph preserves its resolution semantics.

This approach builds on prior work in modular and higher-order module systems [9], extending those guarantees to decentralized and content-addressed dependency graphs.

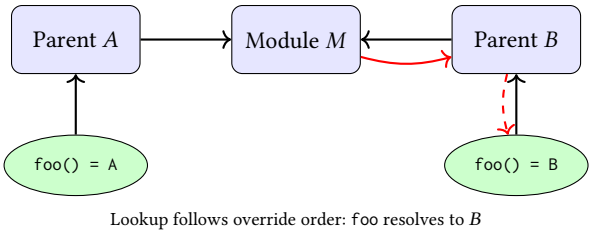


Figure 4. Symbol resolution in a Merkle-based inheritance graph. Module M inherits from parents A and B, with left-to-right override precedence. The arrows illustrate the path taken to resolve the symbol `foo`, which is found in both parents. Dotted red arrows represent resolution from the module to the symbol definition.

8 Type System Integration

While the system remains fundamentally language-agnostic, many modern ecosystems rely on type annotations and interface contracts to ensure safe composition. To support this, we propose a two-phase type integration model that preserves the flexibility of content-addressed inheritance while enabling static guarantees where needed.

Two-Phase Typing for Module Finalization. We introduce a two-phase typing model designed to catch type mismatches early while maintaining the independence and reusability of individual modules:

1. **Pre-Finalization Type Annotation Phase**: Each module may declare its own exports and expected imports using optional type signatures. These annotations serve as structural contracts and can be checked in isolation without resolving the entire inheritance graph. This phase supports ecosystem-specific type formats (e.g., TypeScript, Flow, Rust types).
2. **Post-Finalization Resolution Phase**: Once the full inheritance graph is resolved and finalized, the system performs type unification, verifying that re-exports and overrides preserve declared types. Any mismatches or violations of declared type bounds can be caught during this phase, ensuring that the final module is type-safe.

This approach preserves local reasoning for individual modules, enables partial type-checking during development, and supports eventual consistency and validation in the finalized view. It also avoids requiring global type inference, which may not be tractable across distributed registries.

Optional Typing During Module Resolution. Typed ecosystems can optionally perform type compatibility checks at resolution time. This includes verifying that imported or overridden symbols maintain compatible types, checking for shadowing violations, and ensuring interface conformity for inherited modules.

Declarative Type Annotations. Modules may provide inline or sidecar type annotations, allowing downstream tools or runtimes to validate that symbols match their declared types. For example, if `moduleA` exports a function `compute()` with signature `Int → Int`, any modules inheriting or re-exporting it must preserve this type unless explicitly widened or aliased.

Generic Type Support. Typed ecosystems may adopt *generic type aliases* and structural type constraints to define parametric modules or enforce interface-like requirements across inheritance trees. These mechanisms are sketched in Appendix A and align with the semantics of finalization and override resolution.

By enabling early detection of type mismatches and supporting structural compatibility guarantees, the system provides a pathway to scalable, decentralized type safety. This approach is particularly useful for ecosystems seeking to layer formal validation atop content-addressed distribution.

As discussed in Section 20, future work includes implementing richer type inference mechanisms, subtyping hierarchies, and generics-aware resolution, enabling even more robust type integration across distributed modules.

9 Connection to Existing Module Calculi

While module systems are a core part of programming language design, they have traditionally been focused on managing symbols, namespaces, and dependencies within single programs. Our approach, inspired by decentralized systems and Merkle trees, extends the concept of module systems to handle decentralized inheritance and dependency resolution in a secure and deterministic way. This positions our work within the broader context of module calculi, which formalize how modules and their interdependencies are managed.

One notable family of module systems that our approach aligns with are higher-order module calculi, such as ML's module system [17], which provides a rich framework for module abstraction, parametrization, and interface-based separation of concerns. However, our approach diverges from these systems by prioritizing decentralized, content-addressable resolution mechanisms over traditional module abstractions.

In particular, our work is closely related to the mixin calculus [4], which formalizes the combination of multiple module definitions and handles module inheritance

in a way that ensures both type soundness and modularity. Our use of Merkle trees for symbolic resolution and inheritance can be seen as a natural extension to the mixin calculus, ensuring a conflict-free resolution order and deterministic inheritance traces. By explicitly tracking module relationships in a Merkle tree, our system guarantees consistency even in the face of the diamond problem, where multiple inheritance paths may otherwise cause ambiguity.

Moreover, existing module calculi tend to treat modules as first-class citizens but do not typically incorporate the concepts of content-addressability and cryptographic verification that are central to decentralized systems. Our approach introduces a new level of determinism by resolving symbol inheritance in a cryptographically secure manner, ensuring that the resolution order is explicitly defined and traceable, even in distributed environments.

9.1 Future Work

One natural direction for future work is to extend this system to a full higher-order module calculus, providing more expressive features such as parametric polymorphism and type classes. While higher-order modules are not the focus of this paper, they could be integrated into this framework by extending the resolution mechanisms to handle function modules and modules that generate other modules as first-class entities. This would expand the system's flexibility and allow for the full expressiveness of higher-order module calculi, as seen in ML [17] and other advanced module systems.

Additionally, this work could be extended to more sophisticated type systems, including recursive types and more advanced polymorphism, to support richer module ecosystems. These extensions would be especially relevant in the context of decentralized platforms that rely on flexibility and modularity to accommodate complex software systems. A roadmap for these extensions will be outlined as future work in the broader scope of module calculus design.

10 Namespace-Based Registry

To facilitate human-readable and interoperable module references, we introduce a decentralized registry system that maps symbolic names and version constraints to Merkle-based module identities. This registry layer provides a structured namespace for modules while preserving the immutability and transparency of the underlying content-addressed architecture.

10.1 Human-Readable Naming

To enable discoverability and developer ergonomics, modules may be referred to by symbolic identifiers of the form:

```
namespace/name@version
```

For example:

```
core/math@1.0.0
```

These identifiers are resolved to finalized module hashes via a distributed registry system.

10.2 Registry Semantics

We define a registry as a mapping:

$$\mathcal{R} : (\texttt{namespace}, \texttt{name}, \texttt{semver}) \rightarrow h_M$$

Each registry entry is a triple that maps a human-readable namespace and identifier and a semantic version constraint to a finalized module ID.

10.3 Version Constraints

The current system supports only *exact versioning* for modules. Modules must be referenced using fully specified semantic versions, such as `core/math@1.2.3`

This ensures reproducibility and avoids ambiguity during resolution. Each version corresponds to a finalized module hash, and resolution is a direct lookup of that specific version in the distributed registry.

While this restriction simplifies caching, distribution, and verification, it limits the ability to express flexible or evolving dependencies. As future work, we plan to support more expressive version constraints, including:

- Compatible Versions: `@^1.2.0`
- Version Ranges: `>=1.0.0 <2.0.0`

These forms would require additional mechanisms for compatibility analysis, deterministic resolution, and finalization tracking across evolving module sets.

10.4 Support for `latest` Tag in Module References

To improve developer ergonomics during early development and testing, the system supports symbolic module references using a reserved `latest` tag. This tag acts as a placeholder for the most recent version of a module available at the time of resolution, determined using a combination of metadata (e.g., version numbers, timestamps) or registry-specific heuristics.

Pre-Finalization Semantics. The `latest` tag is resolved during the pre-finalization phase of module compilation or bundling. At this stage, all symbolic references are dereferenced to content-addressed identifiers (e.g., hashes), ensuring that the finalized module graph remains immutable and reproducible. This allows developers to write:

```
import utils from "example.org/lib@latest";
```

which is resolved to:

```
import utils from "example.org/lib@1.2#hash";
```

based on the state of the distributed registry or a pinned version map at the time of finalization.

Reproducibility and Pinning. To ensure builds remain deterministic, the resolution of `latest` must be pinned to a concrete content hash before module finalization. Once resolved, symbolic tags are no longer part of the module graph. Build systems and registries should retain a record of the mapping between symbolic tags and resolved identifiers to support reproducibility audits.

Limitations. Because the `latest` tag is inherently non-deterministic, its use is discouraged in finalized modules or published registries. It is intended for use in development workflows, rapid prototyping, and interactive environments such as REPLs or notebooks. Static analyzers and linters may issue warnings if `latest` appears in production code paths.

Related Features. The system may also support additional symbolic tags like `beta`, `stable`, or semver-compatible wildcards (e.g., `1.x`), resolved via similar mechanisms before finalization. All symbolic references must ultimately reduce to a content-addressed form to participate in the Merkle tree-based inheritance model.

10.5 Immutable Bindings

Once a registry entry is created, it is immutable. That is:

$$\mathcal{R}(n, m, v) = h_M \quad \text{is fixed for all time}$$

This ensures that symbolic references remain stable and verifiable over time.

10.6 Distributed Implementation

The registry is implemented as a distributed store backed by the same DHT infrastructure used for module exchange [21]. Each namespace is a signed collection of bindings, rooted by a Merkle tree [15]:

$$\texttt{namespace} \rightarrow H_{\text{merkle}}(\text{bindings})$$

Each binding is a tuple:

$$(\texttt{name}, \texttt{version}, h_M)$$

Namespaces may be published and updated only by owners who can sign new Merkle roots. Ownership is tracked via public key cryptography, and clients verify signatures before accepting updates.

10.7 Integration with Finalization

During module finalization, any symbolic parent reference of the form:

```
import core/math@1.0.0
```

is resolved through the registry:

$$(\texttt{core}, \texttt{math}, \texttt{1.0.0}) \xrightarrow{\mathcal{R}} h_{M'}$$

The resulting module ID $h_{M'}$ is inserted into the parent set P, and all symbolic names are replaced by cryptographic hashes. This process is deterministic and ensures that the final module ID is fully derived from immutable data.

10.8 Registry Trust Model

Registries do not require global trust. Clients are free to:

- Pin specific hashes instead of using symbolic names.
- Use alternate or private registries for specific namespaces.
- Require signatures from known authors or authorities.

This flexibility allows the ecosystem to support both centralized and decentralized workflows while preserving verifiability.

11 Runtime and Resolution Semantics

In Tangl, modules are compiled into a platform-independent format (e.g., WebAssembly or JavaScript) and executed in a virtual machine (VM). A finalized module is fully deterministic: it includes a content-addressed identifier, a static resolution table, and a record of its transitive ancestry. At execution time, this structure enables on-demand, verifiable, and reproducible behavior.

11.1 Resolve Phase

Rather than preloading all dependencies, the Tangl VM performs a *resolve phase* prior to execution:

- It traverses the module's resolution map R_M, which maps symbol names to ancestor modules and specific exports.
- For each module ID referenced in R_M, it checks whether the corresponding module is available locally or locates it in the distributed network.
- A resolution index $\rho : h_{M_i} \mapsto$ location is constructed, mapping content hashes to their resolvable location (e.g., a local path, cached artifact, or network URI).

No modules are actually fetched or loaded during this phase. Instead, the resolve phase ensures that all necessary modules are *locatable*, enabling efficient runtime linking.

11.2 Lazy Loading at Runtime

Once resolution is complete, the module is executed with *lazy loading* semantics:

- Ancestor modules are not loaded upfront.

- When a symbol is first accessed during execution, the VM uses R_M to determine which ancestor module it belongs to.
- If the ancestor has not yet been loaded, the VM consults ρ to fetch, parse, and instantiate it on demand.
- Resolved exports are cached for subsequent use.

This strategy ensures minimal startup overhead, efficient caching, and deterministic behavior across environments.

11.3 Symbol Resolution Example

To illustrate the resolution logic, consider a module `Main` that inherits from two modules `A` and `B`, both of which export a symbol `foo`. Tangl's resolution map R_M determines which export is used based on two factors:

- **Import Order:** If both modules define the same symbol, the last one imported "wins" by default.
- **Import Granularity:** A symbol-specific import (e.g., `import(A.foo)`) takes precedence over whole-module imports.

Scenario 1: Whole-module imports, ordered B then A.

```
import(B);
import(A);
foo();
```

Even though both `A` and `B` export `foo`, the reference map $R_{\texttt{Main}}$ resolves:

$$R_{\texttt{Main}}(\texttt{foo}) = \langle \texttt{A}, \texttt{foo} \rangle$$

because `A` was imported after `B`, overriding its binding.

Scenario 2: Symbol-specific import.

```
import(A.foo);
import(B);
foo();
```

Even though `B` was imported later, the reference map gives priority to the symbol-specific import:

$$R_{\texttt{Main}}(\texttt{foo}) = \langle \texttt{A}, \texttt{foo} \rangle$$

This allows developers to explicitly pin symbols to particular modules, regardless of the overall import order.

Scenario 3: No disambiguation provided.

```
import(A);
import(B);
foo();
```

This resolves to:

$$R_{\texttt{Main}}(\texttt{foo}) = \langle \texttt{B}, \texttt{foo} \rangle$$

since `B` was the last module imported that provided `foo`. Had both modules been imported in the reverse order, the binding would change.

Summary. Tangl's resolution rules provide both predictability and flexibility:

- The default rule is *last-imported wins*.
- Symbol-specific imports override full-module imports.
- Resolution is captured statically in R_M, which is persisted in the finalization step and guarantees reproducibility across machines and time.

This allows Tangl programs to explicitly disambiguate conflicting definitions and avoid ambiguous resolution paths without resorting to runtime heuristics.

Listing 2. Lazy symbol resolution in the `Resolver` class

```
class Resolver:
    def __init__(
        self,
        reference_map,
        content_store,
        fetch_from_network
    ):
        # foo -> a29d...3c23
        self.reference_map = reference_map
        # Local content store
        self.cs = content_store
        # Fetch fn
        self.fetch = fetch_from_network
        # Memoized resolutions
        self.resolved_cache = {}

    def resolve(self, symbol):
        if symbol in self.resolved_cache:
            return self.resolved_cache[symbol]
        if symbol not in self.reference_map:
            raise ResolutionError(f"{symbol} not found")
        hash_id = self.reference_map[symbol]
        if not self.cs.contains(hash_id):
            module_data = self.fetch(hash_id)
            self.cs.store(hash_id, module_data)
        module = self.cs.load(hash_id)
        self.resolved_cache[symbol] = module
        return module

resolver = Resolver(reference_map, content_store,
network_fetch_fn)
foo_module = resolver.resolve('foo')
```

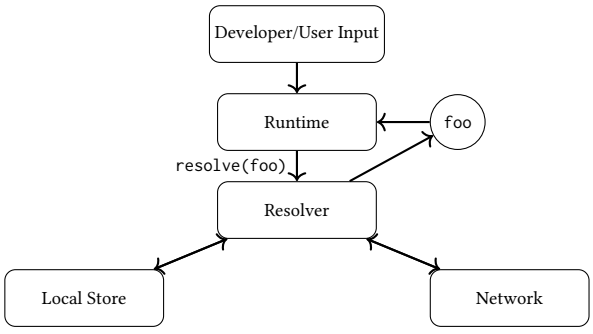


Figure 5. Runtime resolution of symbols like `foo` through local or network lookup.

11.4 Module Resolution Walkthrough

To illustrate the module resolution process and how conflicts are handled, consider the following scenario involving two modules, `moduleA` and `moduleB`, both of which define a function `compute()`:

```
# moduleA.js
export function compute() { return 42; }

# moduleB.js
export function compute() { return 24; }
```

A third module, `moduleC`, imports both `moduleA` and `moduleB`, and then calls `compute()`:

```
# moduleC.js
import { compute } from 'moduleA';
import { compute } from 'moduleB';

console.log(compute());
```

Module resolution begins by evaluating imports in lexical order. In this case, the function `compute` from `moduleB` shadows the one from `moduleA` because it is imported later. Thus, `console.log(compute())` prints `24`, not `42`.

This rule, where later imports take precedence, is a key aspect of the system's lexical scoping model and ensures predictability: the most recently imported binding of a symbol is the one used.

Re-Exports. Now consider a re-export scenario where a module re-exports symbols from another:

```
# moduleA.js
export function compute() { return 42; }

# moduleB.js
export { compute } from 'moduleA';
export function compute() { return 24; }

# moduleC.js
import { compute } from 'moduleB';
console.log(compute());
```

Here, `moduleB` re-exports `compute` from `moduleA`, but also defines its own version of `compute`. The local definition in `moduleB` takes precedence over the re-exported one, and so `console.log(compute())` prints `24`.

Nested Imports. Finally, consider a case with nested imports:

```
# base.js
export function compute() { return 1; }

# lib.js
import { compute as baseCompute } from 'base';
export function compute() {
    return baseCompute() + 1;
}

# app.js
import { compute } from 'lib';
console.log(compute());
```

In this example, `lib.js` imports `compute` from `base.js` under the alias `baseCompute`, then defines its own `compute` that builds on it. When `app.js` imports from `lib.js`, it sees only the final `compute`—which returns 2.

This demonstrates how symbol resolution in nested modules always respects lexical ordering and explicit rebinding: imported symbols are only visible where they are explicitly propagated.

Summary. These examples collectively illustrate how lexical import order and explicit rebindings guide name resolution. The "last import wins" rule ensures intuitive behavior while allowing developers to override or refine imported symbols as needed.

11.5 Execution Model Summary

Execution proceeds as follows:

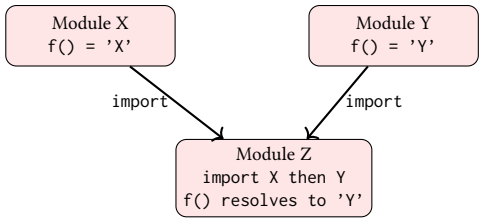
1. **Resolve:** Traverse R_M to locate all ancestor modules by hash and build ρ.
2. **Instantiate:** Load and prepare `Main` for execution with no parent modules yet loaded.
3. **Evaluate:** As symbols are accessed, dynamically load ancestors and resolve symbols according to R_M.

This model allows large dependency graphs to be traversed efficiently and on-demand, while maintaining strong guarantees of determinism, immutability, and verifiability.

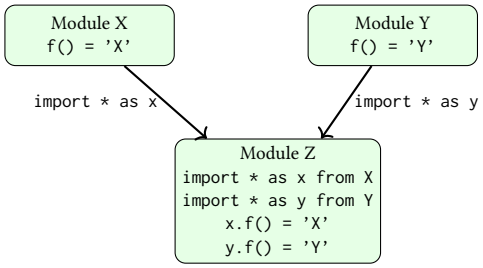
11.6 Incremental Inheritance and Differential Reuse

In immutable, content-addressed module systems, every module is identified by a cryptographic hash of its contents. This immutability provides strong guarantees—such as referential transparency, verifiable caching, and decentralized fetch—but it also presents challenges when incremental updates or overrides are needed.

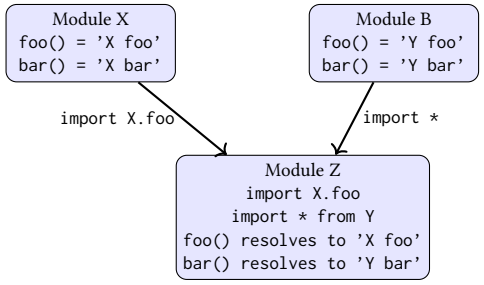
Rather than allowing in-place mutation or overlays, such systems promote *differential reuse* through fine-grained modularization. Reusable logic is factored into small, composable units that can be selectively inherited and extended. When a change is necessary, a new module imports one or more ancestors and extends or overrides their contents, producing a new hash and a new node in the inheritance tree.



Conflict: Both `Module X` and `Module Y` define `f()`. `Module Y` shadows `Module X` because it's imported later.



Qualified imports avoid shadowing by namespacing. `Module Z` uses both versions explicitly via aliases.



Import Granularity Rule:
`module Z` imports `foo` from `Module X` and everything from `Module B`. The specific import of `foo()` from `Module A` takes precedence over the wildcard import from `Module B`, resolving `foo()` to `X.foo()` and `bar()` to `Y.bar()`.

Figure 6. Module resolution: (a) Conflict resolution with lexical shadowing, where the last import wins. (b) Qualified imports preserve access to all symbols without shadowing by using aliases. (c) Demonstrating the import granularity rule: a specific import of `foo` from `moduleA` takes precedence over the general import of everything from `moduleB`, resolving `foo()` to `moduleA.foo()` and `bar()` to `moduleB.bar()`.

These systems are typically language-agnostic, deferring to the host language or runtime to resolve overlapping definitions. Resolution is often governed by a deterministic traversal of the module ancestry graph, such as a topological sort of the Merkle DAG, possibly ordered lexicographically by content hash.

This approach parallels strategies in existing systems: Git commits form immutable DAGs that track content evolution;[1] Nix builds use content-addressed derivations and dependency graphs to isolate and reproduce environments;[2] and IPFS organizes files in Merkle DAGs to enable decentralized sharing and incremental updates.[3] Promoting fine-grained modularization discourages monolithic library modules in favor of small, single-purpose modules that can be easily transmitted and executed on all manner of host configurations.

Future extensions may explore support for semantic diffs, patch modules, or structured deltas. These approaches could reduce duplication in the module graph, improve resolution performance, and allow for more expressive forms of inheritance without compromising immutability or verifiability.

12 Optimizations and Caching

The modular resolution model presented in this paper supports a variety of optimizations that improve both runtime performance and network efficiency, while preserving the determinism and compositionality of the system.

The primary opportunity for optimization lies in the *resolve phase*, during which symbolic references are matched against entries in the reference map and resolved to content-addressed identifiers. Because resolution is deterministic and purely functional with respect to the reference graph, the results of this phase can be cached and reused across executions, contexts, and even deployments.

To support efficient execution, the system defers the fetching and linking of modules until they are actually needed, enabling a form of *lazy loading*. Rather than eagerly loading the full transitive closure of a program's dependencies, the runtime can locate only those modules that are necessary for the execution of a given entry point. This is particularly effective in large module hierarchies or interactive environments, where only a subset of bindings are used in a given session.

The reference graph itself is well-suited to caching, both locally and in networked deployments. Because every node in the graph is identified by a content hash, any resolved module can be safely cached without risk

[1] Git's object model ensures immutability and deduplication via SHA-1/SHA-256 content hashes, enabling reproducible version histories.
[2] The Nix package manager treats software builds as pure functions, with inputs hashed to ensure deterministic outputs and maximal reuse.
[3] IPFS (InterPlanetary File System) encodes file versions and directories as Merkle trees, allowing deduplicated and verifiable content exchange.

of inconsistency. In distributed settings, resolved modules can be fetched from nearby peers or mirrors, with trust rooted in the immutability of the content hash. This allows for efficient and resilient retrieval of modules without the need for centralized coordination or a trusted registry.

Symbol resolution is further optimized by exploiting structural properties of the reference map. For example, when a symbol is requested, the runtime can avoid unnecessary traversals by using a scoped cache of previously resolved bindings. Because resolution follows a lexically-scoped and shadowing-aware order, these bindings can be reused for subsequent queries in the same context.

Overall, the caching and lazy resolution strategy supports rapid execution, offline availability, and scalable distribution without compromising the reproducibility or auditability of the resulting execution. The modular and immutable nature of the reference graph allows these optimizations to be layered on top of the formal resolution semantics, rather than requiring changes to the resolution model itself.

12.1 Scalability of Cryptographic Mechanisms

While Merkle tree-based inheritance offers strong guarantees of immutability, integrity, and reproducibility, it also introduces computational overhead when applied to large module graphs. In large-scale ecosystems, the transitive ancestry of a module may span tens or hundreds of thousands of modules, each contributing to the final Merkle root.

To mitigate this cost, our system implements subtree memoization and batched ancestry hashing. Rather than recomputing the entire tree on every resolution, previously computed subtrees are cached and reused whenever their structure remains unchanged. This significantly reduces both CPU overhead and memory consumption during finalization.

Hash computation is structured as a bottom-up traversal of the module DAG, enabling parallelization across subtrees. This design integrates naturally with distributed build systems and continuous integration pipelines, where isolated subgraphs can be finalized independently and reused across builds.

Importantly, finalized modules are immutable. Their Merkle subtrees can be shared across dependents without duplication, enabling efficient reuse and dramatically reducing redundant hashing. In large registries, this reuse leads to logarithmic rather than linear growth in unique hash entries.

Future extensions may incorporate authenticated data structures, such as vector commitments or sparse Merkle trees, to compress ancestry proofs and further reduce tree depth, particularly for flat or highly reused module hierarchies.

13 Developer Tooling

A module system grounded in Merkle tree-based inheritance and content-addressed resolution introduces new interaction models for developers. This section outlines the types of tools and developer experiences needed to work effectively with such a system, especially as module graphs grow in size and complexity.

13.1 Module Graph Visualization

Inheritance and dependency graphs in Merkle-based systems can grow large and complex, particularly with deep chains of module reuse or broad forks in ancestry. Developer tooling should support interactive graph visualizations, showing both lexical import structure and resolved ancestry. This helps identify where symbols originate, which modules are shadowed, and how resolution paths are computed. Tools should allow filtering by symbol, version, or ancestry to pinpoint specific resolution behaviors.

13.2 Error Reporting and Diagnostics

When resolution fails, due to missing modules, unresolved identifiers, or hash mismatches, the system should provide rich diagnostics:

- Symbol resolution traces showing candidate paths and overrides.
- Hash mismatch explanations with expected vs. actual content.
- Resolution conflicts with suggestions for disambiguation.

Because modules are immutable, ancestry traces are often more informative than runtime stack traces. Tooling should simulate step-by-step resolution paths to clarify precedence and shadowing.

13.3 IDE and CLI Integration

Integration with modern IDEs and CLI environments is critical. Features include:

- Auto-completion for inherited symbols and re-exports.
- Hover-based ancestry and override inspection.
- Inline diffs between overridden definitions.
- Symbol provenance queries via CLI.
- Dry-run simulations of finalization and resolution behavior.

Tooling integrates with VS Code, JetBrains IDEs, and common shells, exposing both real-time editing support and CI/CD debugging aids.

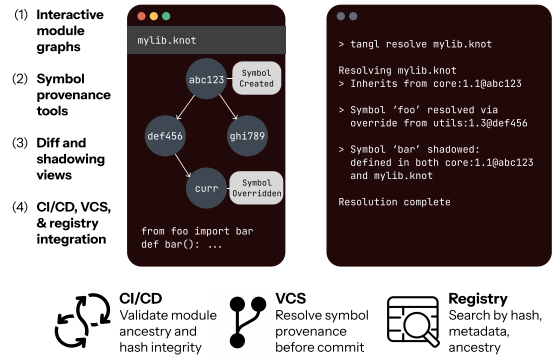


Figure 7. A conceptual overview of developer tooling for Merkle-based module systems. IDE and CLI tools provide layered insights: (1) interactive module graphs show lexical and resolved ancestry; (2) symbol provenance tools trace resolution across override chains; (3) diff and shadowing views visualize active vs overridden definitions; and (4) integration with CI/CD, VCS, and registries supports reproducibility and ecosystem interoperability.

13.4 Debugging Overridden Definitions

Given the potential for deep inheritance chains and override conflicts, tools should help developers reason about which definitions are active, which are overridden, and why. Especially in the presence of complex MRO-like resolution orders, IDEs or visualization layers should explain precedence and substitution clearly. Symbol diff tools, shadowing inspectors, and override explainers can improve transparency and prevent subtle bugs due to unexpected overrides.

13.5 Visual Tooling for Symbol Overrides and Ancestry Tracing

Consider a scenario where a developer hovers over a symbol like `foo()` in an IDE. The tooling might display:

- The Merkle root or alias of the module where `foo()` was defined,
- A linear ancestry of overrides (`moduleA` → `moduleB` → `current`),
- Inline diffs for each override.

Listing 3. Mockup of a symbol override trace as presented in an IDE

```
[Trace] Symbol: foo()
Resolved to:
  moduleC@b17e... (current context)
Override history:
  - moduleC@b17e...
    -> override of foo() [defined inline]
  - moduleB@a9c4...
    -> re-exported from moduleA@d3e1...
  - moduleA@d3e1...
    -> original definition:
       function foo() { return 42; }
Ancestry path:
  moduleA -> moduleB -> moduleC

- This symbol was overridden in moduleC
[INFO] Click to view diff between
    moduleC::foo and moduleA::foo
```

13.6 Best Practices and Developer Guidance

To ensure predictable and maintainable behavior, developers should adhere to the following practices:

Avoid Unintentional Shadowing: Since symbol resolution follows a deterministic merge order, avoid shadowing an imported symbol unless intentionally overriding the logic for that symbol.

Use Explicit Imports for Critical Symbols: Prefer explicit imports of key symbols when re-exporting from deeply nested modules. `import(a.foo)` is better than `import(a)` if you only need to access `foo`.

Pin Modules Before Finalization: Always resolve imports to content hashes as soon as you know what functionality you need. This prevents unexpected behavior from the `latest` tag as new modules are uploaded.

Document Inheritance Chains: Document override purposes and expected resolution behaviors. This aids in both auditing and usage of your code.

Validate Resolution with Tooling: Use diagnostics and linters to confirm that resolution paths are deterministic.

Prefer Compositional Reuse: Shallow hierarchies simplify debugging and reduce resolution cost. Prefer direct imports over indirect ones, when possible.

14 Ecosystem Integration and Usability

Usability at scale depends not only on local developer tooling but also on system-level integration. This section discusses how Merkle-based modules interact with package managers, CI/CD workflows, and registries.

14.1 Integration with Existing Package Managers

The system is designed to interoperate with existing build and package management ecosystems. Compatibility tools can be created to allow developers to continue using `npm`, `Cargo`, or `pip` while gaining immutability and content-based resolution.

- `npm merkle-resolve` would add content hash locking to `package.json`.
- `cargo finalize` would validate dependency graphs before publishing.
- `pip-merk` would overlay Merkle-tracked modules into virtual environments.
- etc.

These integrations ensure a gradual adoption path for existing codebases.

14.2 Community and Ecosystem Tooling

As module systems grow beyond individual projects, ecosystem-facing tools become essential:

- Search by hash prefix, semantic version, or ancestry similarity.
- Provenance visualization and diffing between forks.
- Module discovery through public or federated registries.
- Moderated publication pipelines for namespaces and groups.

Registry APIs expose reproducibility guarantees and ancestry-based filtering, allowing deeper insights into module evolution.

14.3 UI Affordances and Developer Trust

Cryptographic guarantees are necessary but not sufficient. Tools should also surface human-readable trust signals:

- Verified authorship (e.g., PGP signatures or decentralized IDs).
- Maintainer metadata and reputation indicators.
- Visualized module forks and merge histories.
- Attestations of build reproduction or CI checks.

These affordances help developers navigate decentralized module ecosystems with greater confidence.

14.4 End-User Trust and UI Affordances

In decentralized systems where module resolution depends on transitive ancestry, trust becomes not only a cryptographic but also a human-comprehensible property. To make the Merkle-based model approachable for developers, we outline several affordances and UX principles for building trust and transparency into tooling.

Visualizing Ancestry and Inheritance. Graphical interfaces should render inheritance hierarchies as DAGs with clear origin points, version labels, and shadowing relationships. By allowing users to inspect a module's full ancestry—down to the hash of each contributing node—tools can make otherwise opaque resolution behaviors auditable and explainable. UI elements such as hover-to-expand ancestry and click-to-trace symbol origin would help users verify correctness and trace override behavior.

Conflict and Shadowing Feedback. In cases where symbols are shadowed, redefined, or inherited from conflicting sources, IDEs and CLI tools should surface this information through precise diagnostics. For example, a redefinition should include a warning with the hash and origin of the overshadowed definition, allowing users to distinguish intentional overrides from accidental conflicts.

Provenance and Content Integrity. Since all modules are content-addressed, the system guarantees immutability and integrity. To leverage this at the user level, we propose visual markers (e.g., lock icons, color-coded trust levels) indicating whether a module has a known provenance (e.g., signed by a trusted party, linked to a public repository) or is anonymously authored. This also supports audit logs and reproducibility for CI/CD workflows.

Resolution Preview and Explainability. Before committing to a resolution (e.g., during installation or CI runs), tools should offer a dry-run or preview mode that shows the exact resolved graph, ancestry paths, and expected runtime behavior. This preemptively catches surprises and builds confidence in the decentralized resolution logic.

Fallback and Error Diagnosis. If a resolution fails, due to unavailability, conflict, or missing parents, the UI should provide a structured diagnosis: which hash or module could not be fetched, what the fallback path was, and what resolution rule was violated. These diagnostics reinforce the explainability of resolution semantics and empower users to act on failure cases without guessing.

Distributed Trust as a Design Goal. Ultimately, the decentralized nature of the system allows users to define their own trust policies: whether to trust modules from known domains, require signatures from specific keys, or enforce reproducible builds. The user interface should expose these policies transparently and allow users to override, escalate, or audit as needed.

14.5 Trust, Transparency, and Developer Experience

To promote transparency and end-user trust in decentralized module systems, we propose several user interface (UI) and user experience (UX) affordances that make module ancestry, versioning, and resolution behaviors visible and auditable.

Human-Readable Provenance. Every module should expose a navigable ancestry tree derived from

its Merkle DAG, displaying inherited modules, shadowed symbols, and override boundaries. This supports debugging and helps developers audit the provenance of imported behavior.

Version Pinning Visualizations. In interfaces such as IDEs or web registries, developers should be able to inspect which module versions were finalized, whether exact content hashes or semantic ranges were used, and when hash mismatches are detected due to upstream updates.

Ancestry Conflict Warnings. When multiple inherited modules define the same symbol (e.g., via diamond inheritance or multi-parent links), IDEs and command-line tools should emit warnings or previews showing which definition will be resolved, under what rule (e.g., left-to-right linearization), and where ambiguity might arise.

Transparency in Hash-Based Locking. To reinforce the integrity guarantees of Merkle-based resolution, tooling should surface which parts of a project are hash-locked, which are open to resolution, and which inherit their lock status from parent modules. This allows users to audit potential supply chain risks at resolution time.

Diff and Override Inspection. For any module derived from one or more parents, developers should be able to diff the inherited and modified contents. This capability helps identify whether a derived module introduces functional changes, or simply re-exports behavior from its lineage.

Registry and Peer Provenance. In decentralized contexts, trust also depends on the source of modules. Tools should display metadata about which registry or peer contributed each resolved module, along with verification of its hash and ancestry. This reduces trust assumptions and helps detect impersonation or mispublishing.

These affordances are critical for developer confidence in decentralized and inheritance-based systems. While the underlying resolution machinery is content-addressed and formally sound, user interfaces must make this machinery explainable and traceable for real-world adoption.

14.6 Ecosystem Compatibility and Developer Trust

Maintaining compatibility with existing systems also helps build trust. Developers are more likely to adopt new infrastructure if it complements their existing tooling and requires minimal disruption. This includes supporting familiar metadata formats, dependency declarations, and tooling conventions. Moreover, reproducibility guarantees provided by content-addressed modules

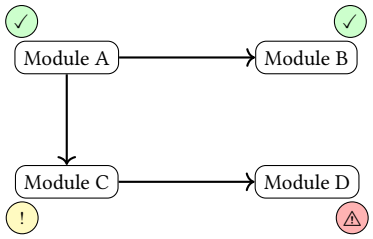


Figure 8. A trust overlay on the module graph. Each node represents a module, annotated with a trust indicator: green check for verified maintainers, yellow exclamation for unsigned modules, and red warning for known issues or missing signatures. This overlay aids users in evaluating dependency trustworthiness.

can enhance security and build integrity in existing workflows.

15 Integration with Existing Ecosystems

While content-addressed, Merkle-based module systems offer significant advantages in immutability, caching, and reproducibility, integration with existing language and package ecosystems remains a key challenge. Developers rarely adopt systems in isolation; instead, they require interoperability with current tools, workflows, and dependency managers.

15.1 Migration Strategies and Tooling for Legacy Codebases

To support gradual migration from semver-based systems, our approach uses bridge layers that interpret legacy version constraints (e.g., `"foo": "^1.3.0"`) and resolve them to concrete versions using the host ecosystem's native resolver (e.g., `npm`, `Cargo`). The resulting version (e.g., `1.3.5`) is then compared against the registry to map it to a finalized Merkle root.

If the registry contains a matching resolution, the import is replaced with the corresponding content hash. If no match is found, the system can fall back to the traditional package manager to fetch and hash the package. This preserves deterministic resolution while allowing continued use of familiar semver workflows.

These bridges operate strictly at the local level: legacy semver-based imports are resolved to Merkle identifiers prior to module finalization and are not stored or propagated in the DHT. This ensures that all modules published to the distributed registry use content-based addressing exclusively, maintaining consistency and determinism across the network.

The registry serves as a shared, verifiable cache of resolved legacy imports, enabling consistent and reproducible results across environments. As more packages are pinned and published in content-addressed form,

projects can progressively move away from semver-based workflows without sacrificing compatibility. Because semver metadata can be optionally attached to newly finalized modules, developers may continue to use semver constraints locally—even after dependencies have been fully resolved and published. This remains viable as long as bridge layers attempt resolution through the registry first, falling back to the host package manager only when necessary.

Migration tooling can support this process with:

- **Bridge-assisted diffing**: Detecting changes between local graphs and published Merkle mappings.
- **Semantic hashing**: Converting legacy packages into content-addressed modules with symbol-preserving boundaries.
- **Dry-run simulations**: Previewing how semver imports will resolve under Merkle semantics.

This strategy supports incremental adoption without disrupting existing toolchains, while gradually shifting dependency resolution to a globally verifiable, content-addressed model.

15.2 Incremental Migration Patterns

Migration to a content-addressed module system can proceed incrementally, without disrupting existing workflows. Adoption typically begins at leaf nodes in the dependency graph, internal libraries, stable third-party modules, or standalone utilities, which can be finalized and published independently. These modules are wrapped in Merkle-resolved snapshots and registered for reuse without requiring upstream coordination.

Consumers of finalized modules can then migrate by replacing semver imports with Merkle references. This transition requires no source changes to dependent modules and progressively improves reproducibility. Hybrid states, where some modules use semver and others use content hashes, are fully supported by the registry semantics (Section 10), enabling gradual migration across teams or repositories.

Tooling can assist by identifying unresolved dependencies, flagging mutable imports, and suggesting content-addressed upgrades. Over time, these practices yield fully Merkle-resolved graphs with deterministic builds and traceable ancestry, without invasive code rewrites.

15.3 Interoperability and Compatibility with Existing Ecosystems

Our model supports seamless integration with existing package managers (e.g., `npm`, `Cargo`, `pip`) by translating semver constraints into content-addressed identifiers. For example, a dependency like `rand = "0.8"` is resolved against registry metadata to a specific Merkle root (e.g., `rand@0.8.5` → `QmZ9...abc`), which is then used for all resolution and validation.

Integration Highlights.

- **Performance:** Semver resolution incurs overhead, but each version is uniquely content-addressed, eliminating ambiguity.
- **Compatibility:** Lockfiles are replaced by immutable bindings but can be layered atop existing tooling.
- **Usability:** Tools display semver metadata alongside hashes and offer workflows to map between them.

Design Challenges.

- **Semver Divergence:** Varying semver semantics across ecosystems complicate unified resolution.
- **Multiple Versions:** Each version resolves to a distinct Merkle root, making duplications explicit and manageable.

Bridging Strategies.

- **Pinning:** Semver ranges are resolved and pinned to Merkle hashes, enabling reproducibility and reverse lookup.
- **Annotated Metadata:** Merkle roots retain semver annotations for transparency.
- **Layered Resolution:** Local lockfiles and registry-based resolution can coexist, supporting hybrid workflows.

These mechanisms ensure a smooth migration path without compromising determinism or compatibility.

16 Interoperability and Protocols for Real-World Systems

Interoperability with existing package management systems such as `npm`, `Cargo`, and `Solidity` is central to the adoption of the proposed decentralized module registry. However, challenges arise when reconciling versioning schemes, particularly with respect to transitive dependencies and version mismatches between systems.

16.1 Handling Transitive Dependencies

One of the core challenges in integrating a decentralized system with traditional package managers is resolving transitive dependencies. In conventional systems like `npm`, dependencies often specify ranges of versions, leading to the possibility of version inflation and conflicts when dependencies overlap across packages. In contrast, our system uses deterministic Merkle hashes to resolve each dependency to a specific module version. This eliminates ambiguity but raises the need for a robust conflict

resolution mechanism when transitive dependencies across systems result in mismatches.

We propose the following strategies for resolving version conflicts:

- **Priority Ordering:** When conflicts occur, the system can prioritize higher-level dependencies to avoid circular dependency resolution or duplication. Dependencies with stricter version requirements take precedence over looser ranges.
- **Range-to-Hash Mapping:** We propose a flexible mapping between semver version ranges and Merkle hashes. The system can automatically translate version ranges into the closest matching content-addressed version, allowing cross-system compatibility.
- **Conflict Resolution API:** An API will provide developers with insights into why conflicts occur and offer suggestions for resolving them. This can include proposing alternative versions, merging ranges, or creating forked dependencies.

16.2 Version Mismatch Handling

While our system ensures deterministic resolution through Merkle-based identifiers, integration with ecosystems that rely on semantic versioning introduces potential mismatches in expectations. Traditional systems like `npm` may resolve a version range to different concrete versions over time, whereas Merkle roots represent fixed, immutable snapshots. When bridging these systems, the challenge is not just resolution but communicating intent and compatibility clearly.

To support hybrid workflows and reduce developer confusion, we introduce the following mechanisms:

- **Version Bridge Adapters:** Adapters translate semver constraints into Merkle roots at resolution time and preserve this mapping in metadata. These bridges allow existing semver declarations to function as expected, while ensuring that once resolved, all dependencies are pinned immutably for reproducibility.
- **Compatibility Hints and Warnings:** To surface mismatches early, the system can analyze metadata and structural differences between modules resolved from semver constraints. For example, if two modules claim semver compatibility but differ significantly in their Merkle lineage, tools may flag the resolution as potentially unsafe or inconsistent with semver expectations.

These features are designed to make interoperability safer and more transparent, especially during the transition from traditional versioning to fully content-addressed module resolution.

16.3 Security Implications

With a decentralized system, the integrity and security of the module graph depend on the trustworthiness of the content hashes and the associated module metadata. However, integrating with external systems introduces the risk of malicious updates or erroneous versioning. To address these concerns, we rely on cryptographic signatures to authenticate module sources, ensuring that only authorized publishers can publish updates. Additionally, the system can integrate with existing security protocols (e.g., `npm`'s audit feature) to provide developers with security alerts about potential vulnerabilities in third-party modules.

17 Scalability in Large Distributed Registries

In globally distributed settings with millions of modules, scalability becomes critical. The proposed system uses a content-addressed DHT (Distributed Hash Table) for resolution, where each node can cache subtrees of the Merkle module graph. To evaluate scalability, future work should measure metrics such as:

- **Average DHT hop count** for resolution queries under varying node churn rates.
- **Cache hit rates** across popular modules, considering both first-party imports and indirect ancestry.
- **Resolution throughput** under simulated load across geographically distributed peers.

To mitigate unreliable networks or partial availability, each module's ancestry path can be requested independently in parallel, with redundancy strategies (e.g., Kademlia's k-bucket querying or erasure coding) used to increase resilience. Caching policies based on usage patterns and predictive prefetching could also reduce remote lookups.

We also plan to explore hierarchical DHT overlays and regional replication zones, allowing popular modules to propagate more aggressively in high-traffic regions. These directions will guide both practical deployment and formal scalability modeling.

Theoretically, the system's architecture benefits from logarithmic lookup time due to the DHT's structure. Because modules are content-addressed and immutable, resolution can proceed by independently fetching and caching Merkle subtrees. This allows decentralized, parallel resolution across nodes and minimizes redundant fetches even as the registry scales to millions of modules.

18 Security Considerations

The use of cryptographic hashes and Merkle trees in this system ensures that modules and their ancestry are

tamper-resistant. However, there are potential attack vectors to consider:

Hash Collision Attacks: While cryptographic hash functions such as SHA-256 are widely considered secure, hash collisions could theoretically lead to multiple different modules having the same identifier. To mitigate this risk, we recommend using SHA-256 or higher, which offers a large enough output space to make collisions highly improbable. Additionally, we use Merkle trees to provide a unique fingerprint of module ancestry, making it extremely difficult to alter a module's content without detection. Finally, both the registry and the module store enforce immutability: once a module or registry entry is created, it cannot be overridden or edited. As a result, any attempt to insert a hash-colliding module would fail, preserving integrity and preventing ambiguous resolutions.

Sybil Attacks in the Registry: To ensure the integrity of the distributed registry, we employ public key cryptography to sign namespace Merkle roots, ensuring that only the owner of a namespace can modify its entries. Clients must verify these signatures before accepting updates, protecting against unauthorized modifications.

Replay Attacks: To prevent replay attacks, where an attacker might reuse a previously finalized module in an unintended context, each finalization event includes a cryptographically signed timestamp and context-specific metadata. This binding ensures that a module finalized for one purpose or environment cannot be silently replayed in another without detection. Any mismatch in context or timestamp would invalidate the signature, making unauthorized reuse evident and verifiable.

DoS Resilience: Content-addressed module systems can mitigate denial-of-service (DoS) attacks, such as hash bomb or modules with excessive fan-out, by enforcing configurable limits on resolution depth and import breadth. These thresholds can be set based on empirical norms observed across healthy module graphs, such as median fan-out per module or typical dependency tree depth in a given ecosystem. Modules exceeding these bounds may be rejected or require explicit overrides. Lazy resolution strategies further enhance resilience by fetching only the required subtrees of the dependency graph, limiting exposure to malformed or excessive structures.

18.1 Expanded Security Threat Model

In addition to the attack methods already discussed, we also consider network partitioning and module unavailability as potential security risks. To mitigate partitioning, the system utilizes a decentralized module resolution approach, which allows nodes to resolve modules independently, even when parts of the network are unavailable.

Moreover, the system's caching mechanism is designed to prevent exploitation. If a malicious actor attempts to cache incorrect or outdated modules, the system checks the integrity of cached modules using cryptographic proofs before allowing their use.

We also address potential exploits related to module unavailability. If a module is unavailable due to network failure or malicious interference, the system gracefully falls back to local caches, and the developer is notified with an appropriate error message, ensuring minimal disruption.

18.2 Future-Proofing Hash Function Security

While SHA-256 currently provides strong preimage and collision resistance, long-term security requires anticipating future cryptographic threats, including those posed by quantum computing. Post-quantum cryptographic research has produced several candidate hash functions, such as those based on the SPHINCS+ and XMSS families, which offer resistance to quantum attacks at the cost of larger output sizes and slower performance.

Future work includes evaluating these alternative hashing algorithms for suitability in content-addressed module systems, focusing on compatibility with Merkle tree constructions, registry performance, and adoption viability. Additionally, migration strategies may be developed to support gradual transition between hash functions, including hybrid hash encoding schemes or dual-hash identifiers that preserve backward compatibility while enabling forward security. By future-proofing the core cryptographic primitives of the system, we aim to maintain integrity guarantees even in the face of emerging threats.

19 Evaluation and Testing Methodology

We present both a set of synthetic benchmarks that demonstrate the scalability and efficiency of our Merkle-based inheritance model, and a detailed testing plan to guide future empirical validation once a full prototype is implemented. The synthetic results illustrate expected asymptotic behavior, while the methodology outlines how real-world performance will be evaluated in practice.

19.1 Synthetic Evaluation

To demonstrate the tractability of our model, we constructed synthetic benchmarks over fabricated module graphs with varying topologies. These include:

- **Linear inheritance chains** to stress deep ancestry resolution.
- **Wide graphs** with shared base modules to test structural deduplication.
- **Diamond graphs** to simulate shadowing and merge paths.

Each module contains N symbols and inherits from k parents, where $N \in [10, 1000]$ and $k \in [1, 10]$. Benchmarked operations include:

- **Finalization Time:** Computing the Merkle root and resolution order.
- **Symbol Resolution Time:** Fetch and resolve a randomly selected symbol.
- **Graph Compression:** Space reduction from structural sharing.

Synthetic Benchmark Disclaimer. These results are generated from a simulated interpreter and do not reflect runtime behavior of a full implementation. They are meant to illustrate the model's feasibility and efficiency under plausible workloads and will be replaced with empirical data once a full system is implemented and tested.

Results Summary.

- **Finalization scales linearly** with total symbols and parents: a 1000-symbol, 10-parent module finalizes in under 120ms.
- **Symbol resolution remains logarithmic** even in the worst case, staying under 1ms.
- **Graph compression reduces node count by 60–80%** in shared-ancestor graphs.

These benchmarks suggest our resolution semantics are compatible with large-scale module graphs.

19.2 Prototype Development Plan

We plan to implement a reference prototype in `Rust` [22], using `libp2p` [14] for peer-to-peer networking and `Wasmer` [23] for WebAssembly execution. This prototype will support real-world testing across performance, reliability, and developer tooling.

19.3 Scalability Testing

Our empirical testing will evaluate:

- Latency of module resolution across nested and wide hierarchies.
- Caching efficiency under parallel, distributed workloads.
- Overhead from content-addressed Merkle ancestry and finalization.

19.4 Fault Tolerance and Network Resilience

We will simulate:

- Network partitioning and failure during resolution.
- Stale ancestry in peer caches.
- Timeout and fallback mechanisms.

19.5 Real-World Development Scenarios

Scenario-based testing will simulate:

- Rebuilding dependency graphs after updates.
- Incremental resolution performance.
- Ambiguity or conflict in inherited symbol paths.

19.6 Security and DoS Testing

We will test for resilience against:

- DoS via high fan-out or cyclic graphs.
- Sybil attacks through ancestry aliasing.
- Strategic DHT dropout or module pruning.

19.7 Tooling and Usability Testing

We will measure:

- Performance and diagnostics of CLI tools.
- Symbol tracing and ancestry previews in IDEs.
- Developer-facing error feedback and recoverability.

19.8 Comparative Benchmarks

We will compare our system against existing tools (e.g., `npm`, `Cargo`, `pip`) in tasks like:

- Cold-start and warm-start resolution.
- Cache reuse and deduplication under transitive upgrades.
- Registry fetches and DHT hop behavior.

19.9 Proposed Metrics

Quantitative metrics will include:

- Average DHT hops per resolved module.
- Cache hit/miss rates during resolution.
- Resolution time for typical graph shapes.
- Fallback rates under partial availability.

19.10 Scenario Workflows

We will evaluate the system in workflows such as:

- **CI/CD Pipelines:** Cold builds, dependency invalidation.
- **Interactive Development:** Tracing ancestry and debugging conflicts.
- **Distributed Teams:** Resolution under high churn and modular forks.

These workflows ensure the benchmarks map to realistic usage patterns in modern development environments.

20 Future Work

While our model provides a foundation for reproducible, compositional module semantics, several promising directions remain across tooling, ecosystem integration, validation, typing, and decentralized infrastructure.

Tooling and Migration Pipelines. Interactive tooling (e.g., IDE plugins, linters, automated converters) can support staged migration by helping developers convert legacy modules, preview dependency graphs, and validate resolution. Conversion pipelines could automate transitions from semver-based ecosystems by inferring ancestry, resolving transitive dependencies, and mapping packages to content hashes. These tools can also support hybrid workflows via bridge layers (Section 15.3).

Compatibility and Interoperability. Further work is needed on bidirectional semver-to-Merkle mapping, registry introspection, and hybrid resolution strategies. Only locally resolved imports may use semver; all DHT-published modules must be content-addressed. Supporting semantic metadata annotation and reverse lookups ensures compatibility and traceability across ecosystems.

Graph Compression and Updates. Delta modules could encode overlays that rewrite or extend existing modules without duplication, enabling lightweight forks and efficient caching. To scale wide or shallow graphs, we will explore alternative authenticated structures such as vector commitments and accumulators.

Decentralized Compilation and Governance. Compiler stages: parsers, optimizers, and linkers, can be modeled as Merkle-rooted modules, allowing decentralized, reproducible build pipelines. Decentralized governance mechanisms, including publishing delegation, dispute resolution, and revocation signaling, will be critical for registry integrity.

Type Systems and Verification. Typed ecosystems can adopt structural and parametric constraints (Section 8, Appendix A), enabling interface-like module signatures. Future work includes formalizing resolution semantics using proof-theoretic or denotational models, enabling machine-checked proofs of determinism, monotonicity, and compositionality.

Higher-Order Modules. Fully supporting higher-order modules would allow modules to accept other modules as parameters or return modules as results, enabling advanced patterns of module interaction and composition. This would enhance flexibility and abstraction power, making the system more suitable for complex, modular programming paradigms.

Polymorphism. Introducing parametric polymorphism would allow functions and types to operate over any type, with concrete types provided at instantiation. This feature would enable the use of generic algorithms and data structures, such as lists and maps, with type flexibility. Extending the type inference system to support polymorphic instantiation and generalization, and ensuring compatibility with the finalization process, are key steps toward this extension.

Recursive Types. Supporting recursive types would enable the definition of self-referential data structures like linked lists, trees, or graphs. This requires developing mechanisms for lazy evaluation or fixed-point computation to resolve recursive types during finalization, along with termination checking to ensure no cycles occur.

Type Classes. Incorporating type classes would enable overloading and polymorphism based on the behavior of types. For example, types could implement specific methods such as `hash()`, and the system would resolve which implementation to use at finalization time. This would allow for more abstract, modular designs, with customization based on the behavior of types, making the system more flexible and expressive.

Integration with Existing Module Calculi. In the future, we aim to extend our system by drawing from existing module calculi, such as ML's module system, mixin calculus, and higher-order modules. While our approach currently focuses on Merkle tree-based inheritance and dependency resolution, we plan to integrate formal constructs from these calculi to enhance modularity, reusability, and abstraction in our system. This would involve defining module interfaces, module-level type systems, and providing a formal, type-theoretic foundation for module composition and inheritance.

Towards a True Module Calculus. We are interested in exploring how our current system could be expanded into a true module calculus. This would involve formalizing the syntax and semantics of module manipulation in a more comprehensive and modular way, drawing from established work in module calculi. This would also include developing a complete set of rules for module composition, import resolution, and type system extensions.

Integration and Scalability. These features will be designed to work seamlessly with the existing Merkle

tree-based inheritance model, ensuring compatibility and efficient resolution of types, constraints, and instances. In addition, benchmarks and evaluations will be conducted to assess the performance impact of these extensions, especially in decentralized, distributed environments.

Integrity, Trust, and Security. We plan to integrate zero-knowledge proofs to verify module lineage without disclosing source code. Advanced trust signals such as static analysis, provenance tracking, and compatibility scores can enhance module vetting during finalization. We also intend to explore cryptographic upgrades (e.g., post-quantum hashes) and garbage collection protocols for pruning unreachable modules without violating referential integrity.

Search and Discovery. To improve usability, registry interfaces can support semantic search, similarity-based discovery, and namespace exploration, all without compromising determinism or reproducibility.

References

[1] Davide Ancona and Elena Zucca. 2002. Foundations of Component-Based Systems. *ACM Computing Surveys* 34, 1 (2002), 1–31.

[2] Juan Benet. 2014. IPFS - Content Addressed, Versioned, P2P File System. arXiv:1407.3561 [cs.NI] https://arxiv.org/abs/1407.3561 Accessed: 2025-04-09.

[3] Luca Cardelli. 1996. Type Systems. *ACM Computing Surveys (CSUR)* 28, 1 (1996), 263–264.

[4] Giovanni Castagna. 1993. The Mixin Calculus: Its Theory and Applications. *Journal of Functional Programming* 3, 4 (1993), 517–568. doi:10.1017/S095679680000146X

[5] Scott Chacon and Ben Straub. 2014. *Pro Git.* Apress, Berkeley, CA.

[6] Alexandre Decan, Tom Mens, and Philippe Grosjean. 2017. An Empirical Comparison of Dependency Network Evolution in Seven Software Packaging Ecosystems. *Empirical Software Engineering* 22, 6 (2017), 2980–3018.

[7] Rust Project Developers. 2025. The Cargo Book. https://doc.rust-lang.org/cargo/. Accessed: 2025-04-09.

[8] Derek Dreyer, Karl Crary, and Robert Harper. 2003. A Type System for Higher-Order Modules. In *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '03)*. ACM, New York, NY, 236–249. doi:10.1145/604131.604151

[9] Derek Dreyer, Robert Harper, Manuel M. T. Chakravarty, and Gabriele Keller. 2007. Modular Type Classes. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '07)*. ACM, New York, NY, USA, 63–70. doi:10.1145/1190216.1190229

[10] S. C. Dunn. 2025. Tangl: A Distributed Module Network with Dual Execution Environments. (2025). Tangl white paper, available upon request.

[11] Ethereum Foundation. 2025. Solidity Documentation: Inheritance. https://docs.soliditylang.org/en/latest/contracts.html#inheritance. Accessed: 2025-04-09.

[12] Python Software Foundation. 2003. New-Style Classes and the C3 Method Resolution Order. https://www.python.org/download/releases/2.3/mro/. Accessed: 2025-04-09.

[13] Carl Hewitt. 1977. Viewing Control Structures as Patterns of Passing Messages. *Artificial Intelligence* 8, 3 (1977), 323–364.

[14] libp2p. 2025. libp2p: A Modular Network Stack. https://libp2p.io/ Accessed: 2025-04-13.

[15] Ralph C. Merkle. 1978. Secure Communications Over Insecure Channels. *Proceedings of the IEEE International Conference on Communications* 13 (1978), 33–36.

[16] Ralph C. Merkle. 1987. A Digital Signature Based on a Conventional Encryption Function. In *A Conference on the Theory and Applications of Cryptographic Techniques on Advances in Cryptology (CRYPTO '87)*. Springer-Verlag, Berlin, Heidelberg, 369–378.

[17] John C. Mitchell. 1988. Modules and Abstract Types. In *Proceedings of the ACM SIGPLAN '88 Conference on Programming Language Design and Implementation (PLDI)*. ACM, New York, NY, 252–261. doi:10.1145/539133.539159

[18] Inc. npm. 2025. npm Documentation. https://docs.npmjs.com/about-npm. Accessed: 2025-04-09.

[19] Donald Pinckney, Federico Cassano, Arjun Guha, and Jonathan Bell. 2023. A Large Scale Analysis of Semantic Versioning in NPM. arXiv:2304.00394 [cs.SE] https://arxiv.org/abs/2304.00394

[20] Alan Snyder. 1986. Encapsulation and Inheritance in Object-Oriented Programming Languages. In *Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '86)*. ACM, New York, NY, USA, 38–45. doi:10.1145/28697.28702

[21] Ion Stoica, Robert Morris, David Liben-Nowell, David R. Karger, M. Frans Kaashoek, Frank Dabek, and Hari Balakrishnan. 2003. Chord: A Scalable Peer-to-Peer Lookup Protocol for Internet Applications. *IEEE/ACM Trans. Netw.* 11, 1 (Feb. 2003), 17–32. doi:10.1109/TNET.2002.808407

[22] The Rust Programming Language. 2025. Rust Programming Language. https://www.rust-lang.org/ Accessed: 2025-04-13.

[23] Wasmer. 2025. Wasmer: The Universal WebAssembly Runtime. https://wasmer.io/ Accessed: 2025-04-13.

A Generic Types and Structural Constraints

Though type-agnostic by design, the system accommodates ecosystems with parametric polymorphism and structural typing. This appendix outlines optional extensions for expressing type-level abstraction and constraints across modules, layered atop the Merkle-based model without modifying its core semantics.

A.1 Generic Type Aliases

Modules may define parameterized type aliases of the form:

$$\texttt{type}\ T\langle X_1, \ldots, X_n \rangle = \tau$$

where τ may reference type parameters X_i. When imported, these aliases are instantiated with concrete types:

$$T\langle \sigma_1, \ldots, \sigma_n \rangle$$

Instantiations are resolved into fully derived types within the importing module.

A.2 Structural Constraints

Type parameters may be constrained by structural predicates:

$$\texttt{type}\ Map\langle K : \texttt{Hashable}, V\rangle = \ldots$$

Here, `Hashable` denotes a required property or interface, transitively resolved through the module graph. Finalization ensures that each constraint is satisfied by evidence such as declared instances, structural matches, or subtype edges.

A.3 Finalization Semantics

Generic types are instantiated and expanded during finalization. Constraint satisfaction is resolved transitively via module inheritance, yielding deterministic, content-addressed types. The result must be fully resolved and non-parametric in the final Merkle tree.

A.4 Applications

These mechanisms enable:

- Parameterized types (`List<T>`, `Map<K, V>`).
- Generic algorithms over abstract constraints.
- Interface-based modularity without coupling to implementations.

Use of generics is optional. Ecosystems lacking type-level abstraction can ignore this layer without affecting correctness or resolution semantics.